

```
C:\WINDOWS\system32\cmd. x + v

@app.get("/")
def root():
    return {
        "message": "W12D3 ML Monitoring API is running",
        "status": "healthy",
    }

@app.get("/health")
def health():
    return {
        "status": "healthy",
        "model": "RandomForestClassifier",
    }

@app.post("/predict")
def predict(request: PredictionRequest):
    features = [
        request.sepal_length,
        request.sepal_width,
        request.petal_length,
        request.petal_width,
    ]

    prediction = int(model.predict(features)[0])

    return {
        "prediction": prediction,
    }

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>ruff check .
All checks passed!
```

```
C:\WINDOWS\system32\cmd. x + v

    return {
        "prediction": prediction,
    }
}

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>ruff check .
All checks passed!

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>python -m pytest -q
... [100%]
===== warnings summary =====
.venv\Lib\site-packages\fastapi\testclient.py:1
C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring\.venv\Lib\site-packages\fastapi\testclient.py:1:
StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 1 warning in 17.75s

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker build -t w12d3-ml-monitoring:latest
[+] Building 0.3s (1/1) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.1s
=> => transferring dockerfile: 2B                               0.0s
ERROR: failed to build: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory

What's next:
  Debug this build failure with Gordon → docker ai "help me fix this build failure"

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>dir
Volume in drive C is Windows-SSD
Volume Serial Number is EA8D-9B22

Directory of C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring

28-09-2026  19:52    <DIR>          .
28-09-2026  19:21    <DIR>          ..
28-09-2026  19:32                89 .dockerignore
```

```
C:\WINDOWS\system32\cmd. x + v
(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker build -t w12d3-ml-monitoring:latest
[+] Building 139.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 323B
=> [internal] load metadata for docker.io/library/python:3.12-slim
=> [internal] load .dockerignore
=> => transferring context: 129B
=> [1/5] FROM docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
=> => resolve docker.io/library/python:3.12-slim@sha256:f77ac9e44ae96ef2c90b8053ea08c31f8be030f824196b0ae4db6d462c84e51f
=> [internal] load build context
=> => transferring context: 3.90kB
=> CACHED [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY app ./app
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:95534fa18595097d0976d8f5fbc7a25ac54aa641400e28fda5ea2855e8ee511e
=> => exporting config sha256:e0f344b8f65ce6d3cfb8318f28c092f4a6d2d2f8afe781f6ad47af82cbd0285a
=> => exporting attestation manifest sha256:9f5390acc90a911a6a186fa1a4852b41730e2f27b50b5ad6c7be8d1c65a1e204
=> => exporting manifest list sha256:76148c63f2a2f42d3f867e5744d5b77c5f9bf2c74175a0f9f9c5c55cf4cfa6f0
=> => naming to docker.io/library/w12d3-ml-monitoring:latest
=> => unpacking to docker.io/library/w12d3-ml-monitoring:latest

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker images
INFO In Use
IMAGE ID DISK USAGE CONTENT SIZE EXTRA
mlops-ml-api:latest 068632f888ea 627MB 147MB U
production-rag-api:latest ca25408bcb55 310MB 75.8MB U
w12d2-ml-api:latest 6bbb1d5f399d 604MB 140MB
w12d3-ml-monitoring:latest 76148c63f2a2 591MB 137MB

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker run -d --name w12d3-ml-monitoring-c
ontainer -p 8000:8000 w12d3-ml-monitoring:latest
fab62e63b7ae656ed85551ac342f2b9b2e145661d2aa8814320d840a7504b81b
```

```
C:\WINDOWS\system32\cmd. x + v
(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>type MONITORING.md
\# Production ML Model Monitoring Strategy

\## 1. Objective

The objective of this monitoring strategy is to maintain the reliability, availability, and performance of the ML API after deployment.

The monitoring strategy covers:

\ - API and system health
\ - Model performance
\ - Input data quality
\ - Data drift
\ - Alerts
\ - Model retraining triggers

\## 2. API and System Metrics
```

```
C:\WINDOWS\system32\cmd. x + v

The following production metrics should be monitored:

\ - API uptime and availability
\ - Number of requests
\ - Request latency
\ - HTTP 4xx and 5xx errors
\ - Container health
\ - CPU usage
\ - Memory usage
\ - Application failures

These metrics help identify service availability and infrastructure problems.

\## 3. Model Performance Metrics

The following model metrics should be tracked when labeled production data becomes available:
```

```
C:\WINDOWS\system32\cmd. x + v

\ - Accuracy
\ - Precision
\ - Recall
\ - F1-score
\ - Prediction distribution
\ - Model confidence, when available

Production performance should be compared with validation performance to identify degradation.

\## 4. Data Quality Monitoring

Production input data should be checked for:

\ - Missing values
\ - Invalid values
\ - Unexpected feature ranges
\ - Malformed requests
\ - Unexpected categorical values
```

```
C:\WINDOWS\system32\cmd. x + v

\ - Unexpected categorical values
\ - Duplicate or corrupted records

Data-quality problems should be investigated before using affected data for retraining.

\## 5. Data Drift Monitoring

Data drift occurs when the distribution of production input data changes compared with the training or validation data.

The monitoring system should track:

\ - Feature means and standard deviations
\ - Feature distributions
\ - Categorical value frequencies
\ - Missing-value rates
\ - Significant changes in input distributions

Persistent and significant drift should trigger an investigation.
```

```
C:\WINDOWS\system32\cmd. x + v

A single anomalous observation should not automatically trigger model retraining.

\## 6. Alert Conditions

Alerts should be configured for important production problems such as:

\ - API becoming unavailable
\ - Increased HTTP 5xx errors
\ - High request latency
\ - Container failure
\ - Excessive CPU or memory usage
\ - High invalid-input rate
\ - Significant data drift
\ - Unexpected prediction distribution
\ - Model performance falling below the defined threshold

\## 7. Model Retraining Triggers
```

```
C:\WINDOWS\system32\cmd. x + v

Model retraining should be considered when:

\ - Model performance consistently falls below the defined threshold
\ - Significant data drift persists
\ - Production data distribution changes substantially
\ - Unexpected prediction behavior is observed
\ - Sufficient new labeled data becomes available
\ - Business or application requirements change

Retraining should not be triggered solely by one unusual prediction or one isolated data point.

\## 8. Retraining Workflow

The proposed retraining workflow is:

1\ . Collect new production data.
2\ . Validate data quality.
3\ . Check for data drift.
```

```
C:\WINDOWS\system32\cmd. x + v

3\ . Check for data drift.
4\ . Prepare and preprocess the data.
5\ . Train a candidate model.
6\ . Evaluate the candidate model.
7\ . Compare it with the current production model.
8\ . Track the experiment using MLflow.
9\ . Register the candidate model if it satisfies the required criteria.
10\ . Run automated tests.
11\ . Deploy after validation.
12\ . Continue monitoring the new production model.

\## 9. Tools Used

The W12D3 project uses the following technologies:

\ - Python
\ - FastAPI
\ - Scikit-learn
\ - Docker
```

```
C:\WINDOWS\system32\cmd. x + v

\~ Pytest
\~ Ruff
\~ Git
\~ GitHub
\~ GitHub Actions
\~ Docker Hub
\~ MLflow
\~ MLOps practices

\## 10. Monitoring Review

Monitoring metrics and thresholds should be reviewed regularly.

Operational alerts should be investigated before taking corrective action, and retraining decisions should be based on sustained evidence of model or data degradation.

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker stop w12d2-ml-api-container
w12d2-ml-api-container

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker rm w12d3-ml-monitoring-container
w12d3-ml-monitoring-container
```

```
C:\WINDOWS\system32\cmd. x + v

Monitoring metrics and thresholds should be reviewed regularly.

Operational alerts should be investigated before taking corrective action, and retraining decisions should be based on sustained evidence of model or data degradation.

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker stop w12d2-ml-api-container
w12d2-ml-api-container

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker rm w12d3-ml-monitoring-container
w12d3-ml-monitoring-container

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker run -d --name w12d3-ml-monitoring-container -p 8000:8000 w12d3-ml-monitoring:latest
fb880307ec5c8f0f8d38ce590da0aaa7589bfa3893deb4e6d0af4963e5893a7e

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
fb880307ec5c   w12d3-ml-monitoring:latest          "uvicorn app.main:ap..." 8 seconds ago  Up 7 seconds  0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
w12d3-ml-monitoring-container

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>notepad MONITORING.md

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>type MONITORING.md
\# Production ML Model Monitoring Strategy

\## 1. Objective

The objective of this monitoring strategy is to maintain the reliability, availability, and performance of the ML API after deployment.
```

```
C:\WINDOWS\system32\cmd. x + v

The objective of this monitoring strategy is to maintain the reliability, availability, and performance of the ML API after deployment.

The monitoring strategy covers:

\ - API and system health
\ - Model performance
\ - Input data quality
\ - Data drift
\ - Alerts
\ - Model retraining triggers

\## 2. API and System Metrics

The following production metrics should be monitored:

\ - API uptime and availability
\ - Number of requests
\ - Request latency
```

```
C:\WINDOWS\system32\cmd. x + v

\ - Request latency
\ - HTTP 4xx and 5xx errors
\ - Container health
\ - CPU usage
\ - Memory usage
\ - Application failures

These metrics help identify service availability and infrastructure problems.

\## 3. Model Performance Metrics

The following model metrics should be tracked when labeled production data becomes available:

\ - Accuracy
\ - Precision
\ - Recall
\ - F1-score
\ - Prediction distribution
\ - Model confidence, when available
```

```
C:\WINDOWS\system32\cmd. x + v

Production performance should be compared with validation performance to identify degradation.

\\## 4. Data Quality Monitoring

Production input data should be checked for:

\\- Missing values
\\- Invalid values
\\- Unexpected feature ranges
\\- Malformed requests
\\- Unexpected categorical values
\\- Duplicate or corrupted records

Data-quality problems should be investigated before using affected data for retraining.

\\## 5. Data Drift Monitoring

Data drift occurs when the distribution of production input data changes compared with the training or validation data.
```

```
C:\WINDOWS\system32\cmd. x + v

The monitoring system should track:

\\- Feature means and standard deviations
\\- Feature distributions
\\- Categorical value frequencies
\\- Missing-value rates
\\- Significant changes in input distributions

Persistent and significant drift should trigger an investigation.

A single anomalous observation should not automatically trigger model retraining.

\\## 6. Alert Conditions

Alerts should be configured for important production problems such as:

\\- API becoming unavailable
\\- Increased HTTP 5xx errors
\\- High request latency
```

```
C:\WINDOWS\system32\cmd. x + v

\ - Container failure
\ - Excessive CPU or memory usage
\ - High invalid-input rate
\ - Significant data drift
\ - Unexpected prediction distribution
\ - Model performance falling below the defined threshold

## 7. Model Retraining Triggers

Model retraining should be considered when:

\ - Model performance consistently falls below the defined threshold
\ - Significant data drift persists
\ - Production data distribution changes substantially
\ - Unexpected prediction behavior is observed
\ - Sufficient new labeled data becomes available
\ - Business or application requirements change
```

```
C:\WINDOWS\system32\cmd. x + v

Retraining should not be triggered solely by one unusual prediction or one isolated data point.

## 8. Retraining Workflow

The proposed retraining workflow is:

1\ . Collect new production data.
2\ . Validate data quality.
3\ . Check for data drift.
4\ . Prepare and preprocess the data.
5\ . Train a candidate model.
6\ . Evaluate the candidate model.
7\ . Compare it with the current production model.
8\ . Track the experiment using MLflow.
9\ . Register the candidate model if it satisfies the required criteria.
10\ . Run automated tests.
11\ . Deploy after validation.
12\ . Continue monitoring the new production model.
```

```
C:\WINDOWS\system32\cmd. x + v

\## 9. Tools Used

The W12D3 project uses the following technologies:

\ - Python
\ - FastAPI
\ - Scikit-learn
\ - Docker
\ - Pytest
\ - Ruff
\ - Git
\ - GitHub
\ - GitHub Actions
\ - Docker Hub
\ - MLflow
\ - MLOps practices

\## 10. Monitoring Review
```

```
C:\WINDOWS\system32\cmd. x + v

\## 10. Monitoring Review

Monitoring metrics and thresholds should be reviewed regularly.

Operational alerts should be investigated before taking corrective action, and retraining decisions should be based on sustained evidence of model or data degradation.

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>powershell -Command "$p=Get-Content MONITORING.md -Raw; $p=$p -replace '\\#','#' -replace '\\##','##' -replace '\\-','- ' -replace '\\\[0-9]+\.'','$1.'; Set-Content MONITORING.md $p"

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>type MONITORING.md
# Production ML Model Monitoring Strategy

## 1. Objective

The objective of this monitoring strategy is to maintain the reliability, availability, and performance of the ML API after deployment.

The monitoring strategy covers:

- API and system health
```

```
C:\WINDOWS\system32\cmd. x + v
- API and system health
- Model performance
- Input data quality
- Data drift
- Alerts
- Model retraining triggers

## 2. API and System Metrics

The following production metrics should be monitored:

- API uptime and availability
- Number of requests
- Request latency
- HTTP 4xx and 5xx errors
- Container health
- CPU usage
- Memory usage
- Application failures
```

```
C:\WINDOWS\system32\cmd. x + v

These metrics help identify service availability and infrastructure problems.

## 3. Model Performance Metrics

The following model metrics should be tracked when labeled production data becomes available:

- Accuracy
- Precision
- Recall
- F1-score
- Prediction distribution
- Model confidence, when available

Production performance should be compared with validation performance to identify degradation.

## 4. Data Quality Monitoring

Production input data should be checked for:
```

```
C:\WINDOWS\system32\cmd. x + v

- Missing values
- Invalid values
- Unexpected feature ranges
- Malformed requests
- Unexpected categorical values
- Duplicate or corrupted records

Data-quality problems should be investigated before using affected data for retraining.

## 5. Data Drift Monitoring

Data drift occurs when the distribution of production input data changes compared with the training or validation data.

The monitoring system should track:

- Feature means and standard deviations
- Feature distributions
- Categorical value frequencies
```

```
C:\WINDOWS\system32\cmd. x + v

- Categorical value frequencies
- Missing-value rates
- Significant changes in input distributions

Persistent and significant drift should trigger an investigation.

A single anomalous observation should not automatically trigger model retraining.

## 6. Alert Conditions

Alerts should be configured for important production problems such as:

- API becoming unavailable
- Increased HTTP 5xx errors
- High request latency
- Container failure
- Excessive CPU or memory usage
- High invalid-input rate
- Significant data drift
```

```
C:\WINDOWS\system32\cmd. x + v

- Unexpected prediction distribution
- Model performance falling below the defined threshold

## 7. Model Retraining Triggers

Model retraining should be considered when:

- Model performance consistently falls below the defined threshold
- Significant data drift persists
- Production data distribution changes substantially
- Unexpected prediction behavior is observed
- Sufficient new labeled data becomes available
- Business or application requirements change

Retraining should not be triggered solely by one unusual prediction or one isolated data point.

## 8. Retraining Workflow
```

```
C:\WINDOWS\system32\cmd. x + v

The proposed retraining workflow is:

1\. Collect new production data.
2\. Validate data quality.
3\. Check for data drift.
4\. Prepare and preprocess the data.
5\. Train a candidate model.
6\. Evaluate the candidate model.
7\. Compare it with the current production model.
8\. Track the experiment using MLflow.
9\. Register the candidate model if it satisfies the required criteria.
10\. Run automated tests.
11\. Deploy after validation.
12\. Continue monitoring the new production model.

## 9. Tools Used

The W12D3 project uses the following technologies:
```

```
C:\WINDOWS\system32\cmd. x + v

- Python
- FastAPI
- Scikit-learn
- Docker
- Pytest
- Ruff
- Git
- GitHub
- GitHub Actions
- Docker Hub
- MLflow
- MLOps practices

## 10. Monitoring Review

Monitoring metrics and thresholds should be reviewed regularly.

Operational alerts should be investigated before taking corrective action, and retraining decisions should be based on sustained evid
```

```
C:\WINDOWS\system32\cmd. x + v

Operational alerts should be investigated before taking corrective action, and retraining decisions should be based on sustained evidence of model or data degradation.

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>ruff check .
All checks passed!

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>python -m pytest -q
[100%]
===== warnings summary =====
.venv\Lib\site-packages\fastapi\testclient.py:1
C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring\.venv\Lib\site-packages\fastapi\testclient.py:1:
StarletteDeprecationWarning: Using 'httpx' with 'starlette.testclient' is deprecated; install 'httpx2' instead.
  from starlette.testclient import TestClient as TestClient # noqa
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
3 passed, 1 warning in 18.59s

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>notepad .github\workflows\ci.yml

(.venv) C:\Users\Sanjana CA\OneDrive\Documents\CynarisInternship\W12D3_ML_Model_Monitoring>type .github\workflows\ci.yml
name: W12D3 ML Monitoring CI/CD

on:
  push:
    branches:
      - main
      - "feat/**"
  pull_request:
    branches:
      - main

jobs:
  lint:
    runs-on: ubuntu-latest
```


Complete MLOps To x Iris ML Prediction AI x Iris ML Prediction AI x localhost:8000 x W12D3: Monitoring x + Ask Gemini

localhost:8000/docs#/default/predict_predict_post

GET /health Health

Parameters Cancel

No parameters

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:8000/health' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:8000/health
```

Server response

Code	Details
200	Response body <pre>{ "status": "healthy" }</pre> Download

Response headers

```
content-length: 20
content-type: application/json
date: Mon, 28 Sep 2026 14:46:07 GMT
server: uvicorn
```

Complete MLOps To x Iris ML Prediction AI x Iris ML Prediction AI x localhost:8000 x W12D3: Monitoring x + Ask Gemini

localhost:8000/docs#/default/predict_predict_post

Response headers

```
content-length: 20
content-type: application/json
date: Mon, 28 Sep 2026 14:46:07 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

POST /predict Predict

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
```

Complete MLOps To... Iris ML Prediction AI... localhost:8000... W12D3: Monitoring... Ask Gemini

localhost:8000/docs#/default/predict_predict_post

POST /predict Predict

Parameters Cancel Reset

No parameters

Request body required application/json

Edit Value | Schema

```
{
  "sepal_length": 5.1,
  "sepal_width": 3.5,
  "petal_length": 1.4,
  "petal_width": 0.2
}
```

Execute Clear

Responses

Curl

Complete MLOps To... Iris ML Prediction AI... localhost:8000... W12D3: Monitoring... Ask Gemini

localhost:8000/docs#/default/predict_predict_post

Responses

Curl

```
curl -X 'POST' \
  'http://localhost:8000/predict' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "sepal_length": 5.1,
    "sepal_width": 3.5,
    "petal_length": 1.4,
    "petal_width": 0.2
  }'
```

Request URL

<http://localhost:8000/predict>

Server response

Code	Details
200	Response body <pre>{ "prediction": 0, "class_name": "setosa" }</pre> Response headers <pre>content-length: 38 content-type: application/json date: Mon, 28 Sep 2026 14:39:57 GMT server: unicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type application/json

Complete MLOps To... Iris ML Prediction AI... Iris ML Prediction AI... localhost:8000... W12D3: Monitoring... Ask Gemini

localhost:8000/docs#/default/predict_predict_post

200 Successful Response No links

Media type
application/json

Controls Accept header.

Example Value | Schema

```
"string"
```

422 Validation Error No links

Media type
application/json

Example Value | Schema

```
{
  "detail": [
    {
      "loc": [
        "string",
        0
      ],
      "msg": "string",
      "type": "string",
      "input": "string",
      "ctx": {}
    }
  ]
}
```

Schemas

HTTPValidationError > Expand all object

Complete MLOps To... Iris ML Prediction AI... Iris ML Prediction AI... localhost:8000... W12D3: Monitoring... Ask Gemini

localhost:8000

```
{
  "message": "Iris ML API is running"
}
```

Raw Parsed